

TPL POO : Simulation des robots pompiers.

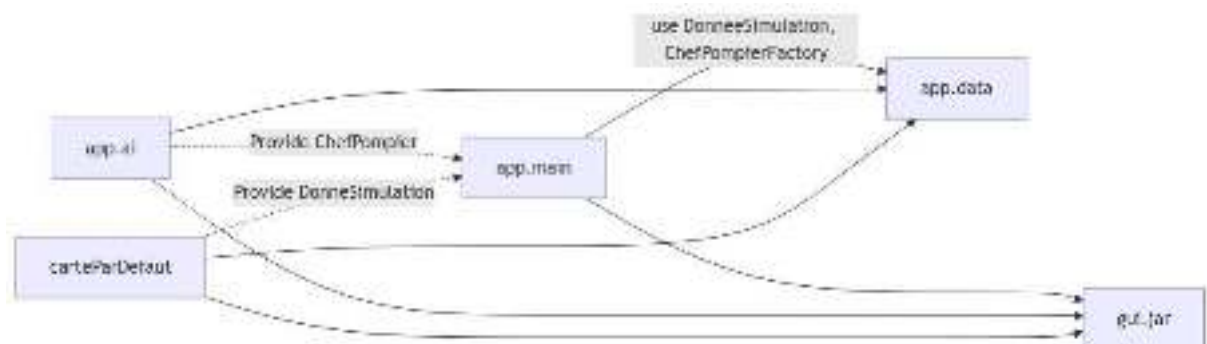
I. Introduction :

Ce projet vise à développer en Java une application permettant de simuler une équipe de robots pompiers évoluant de manière autonome dans un environnement naturel. Cette application se concentre sur la gestion de l'accessibilité de chaque robot aux différentes cellules de la carte, leurs interventions et le calcul des trajets optimaux en fonction de leur type et des restrictions imposées par le terrain.

Note: - Pour pouvoir programmer avec les fonctionnalités récentes de java, on a dû modifier légèrement gui.jar (pas le code) au niveau du MANIFEST.MF pour donner un module par défaut à notre jar, utilisez donc le jar fourni sinon l'application ne marchera pas. Pour reproduire notre manipulation, il suffit de créer un fichier MANIFEST.MF dans lequel on colle la ligne : "Automatic-Module-Name: gui" (sans les guillemets) et lancer la commande: "jar -umf MANIFEST.MF gui.jar" à l'endroit où se trouve gui.jar. Vous pouvez lancer directement ça: echo "Automatic-Module-Name: gui" > MANIFEST.MF && jar -umf MANIFEST.MF gui.jar Sans cette petite manipulation (qui ne coûte rien et ne change pas le code de gui.jar) on est contraint de se passer du système de module que l'on a utilisé abondamment dans ce projet.

- Les textures des robots ont été générées par l'intelligence artificielle (L'IA a été utilisée uniquement pour générer les textures des robots).

II. Structure du projet :



Le projet est structuré en plusieurs modules, chacun ayant un rôle spécifique pour assurer une séparation claire des tâches et faciliter la maintenance du code.

- Module **app** : Il contient le point d'entrée principal (*main*) de l'application. Ce module est responsable de l'initialisation de l'environnement de simulation.
- Module **app.data** : Il regroupe plusieurs packages :
 - **org.app.ai** : ce package contient l'algorithme du plus court chemin et la structure Noeud.
 - **org.app.data** : ce package contient toutes les données fondamentales au fonctionnement de l'application comme la classe Robot et ses sous-classes.
 - **org.app.events** : ce package gère les événements (comme les déplacements des robots, le remplissage du réservoir, etc.)

- *org.app.exceptions* : ce package regroupe les classes d'exceptions spécifiques pour gérer les erreurs ou les conditions exceptionnelles au sein de la simulation.
- *org.app.io* : ce package est responsable des interactions d'entrée/sortie, comme la lecture des configurations de la carte.
- Module *app.test* : Il contient des tests pour valider les fonctionnalités et les comportements de l'application.
- Module *app.ai*: Contient l'implémentation des chefs de pompiers.

III. Organisation des classes et Implémentation :

1. Première partie du projet :

Le diagramme UML ci-dessous représente la structure des classes principales utilisées pour la gestion des données dans la simulation.

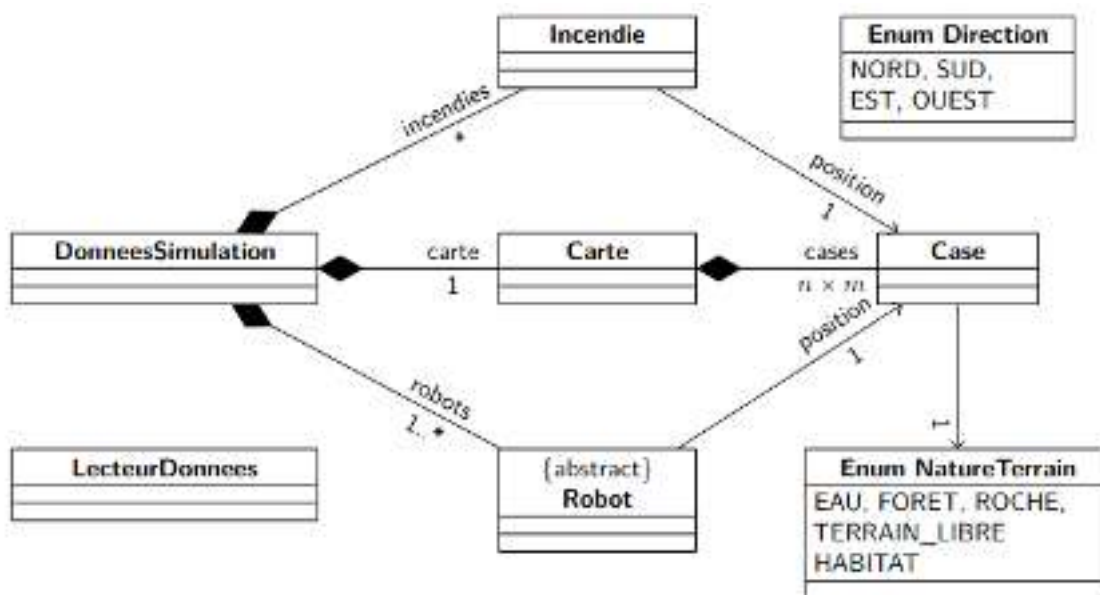


FIGURE 1 – Diagramme de classes des données du problème.

Un module Java 9 nommé : “CarteParDefaut” est chargé dynamiquement au lancement du programme et fournit les cartes via des implémentations de DonneesSimulation.

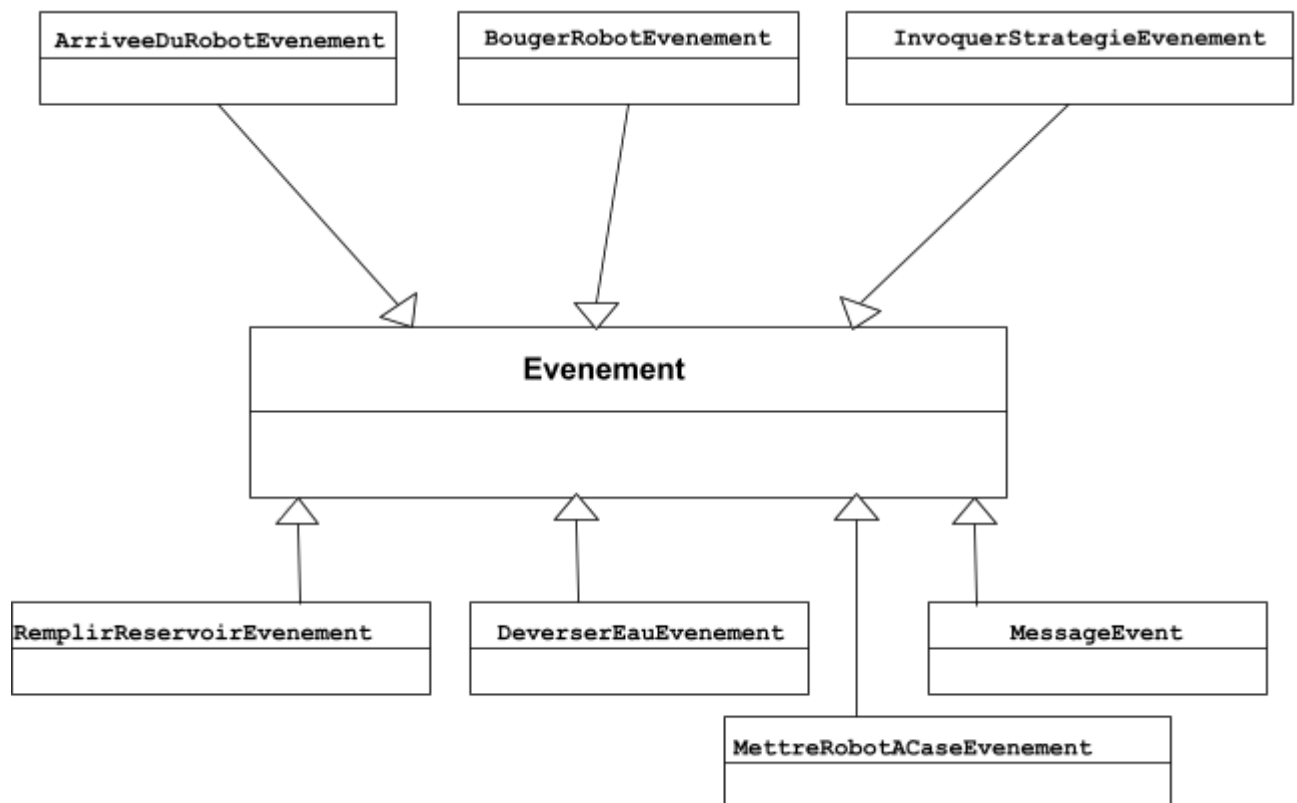
Comme le chargement est dynamique l'application on peut modder/étendre l'application avec un module que vous aurez codé et qui ne fait pas partie de notre implémentation de base. Pour cela il suffit juste de faire un module fournissant des classes DonneeDeSimulation (voir notre code et surtout le module-info.java).

2. Deuxième partie du projet :

Nous avons défini une classe de base abstraite **Evenement**, dont chaque type d'événement spécifique (comme **BougerRobotEvenement**, etc.) hérite. Chaque sous-classe représente un événement précis avec une méthode **execute()** qui définit son comportement.

Nous avons de même défini plusieurs exceptions dans le package `org.app.exceptions` dans le module `app.data` de telle sorte que chaque exception spécifique hérite du `RuntimeException`.

Ci-dessous un diagramme UML pour mettre en évidence l'organisation des classes dans la partie 2 pour le gestionnaire d'événements discrets :



3. Troisième partie du projet :

Dans cette partie, pour calculer les plus courts chemins, on a choisi l'algorithme **A*** de complexité variante selon le choix de l'heuristique. En particulier, avec une heuristique nulle, on retombe sur Dijkstra (de complexité spatiale $O(V)$ et de complexité temporelle $O((V+E)\log(V))$).

Notre idée consiste à explorer les voisins au fur et à mesure à partir de la case de départ. Ensuite, nous transformons les cases voisines en nœuds. On a défini un nœud comme une structure qui contient la case et le temps nécessaire pour arriver à cette case à partir de la case de départ. Le coût entre deux cases(nœuds) voisines **gCost** est le temps nécessaire pour aller du milieu de la première case au milieu de la seconde case. Nous avons opté pour cette approche car d'une part, nous parcourons le minimum nécessaire de cases pour trouver le chemin optimal et d'autre part la complexité de l'algorithme est satisfaisante.

À cet égard, nous avons implémenté deux classes dans le package org.app.ai : une première classe statique qui contient l'algorithme du plus court chemin. Ainsi, il suffit de remplacer A* dans cette classe par un autre algorithme de plus court chemin si besoin. Et une deuxième classe qui représente la structure Noeud.

4. Les stratégies

L'implémentation des stratégies de base se font dans le module app.ai. Le module app.ai fournit une classe ChefPompierFactory pour l'application. Dans la conception de la bibliothèque, de même que pour les cartes, on charge dynamiquement la stratégie. Donc il est possible de modifier l'application avec des stratégies non implémentées par nous. Il suffit de faire un module fournissant des classes ChefPompierFactory, voir app.ai pour le fonctionnement (et la compilation des modules en Java 9). Par défaut on a les deux stratégies de base portant le nom de : "ChefElementaire" et "ChefElementaireAmeliore".

Les stratégies sont implémentées de manière purement événementielles. C'est à dire que le robot ne reçoit pas réellement un événement mais plutôt un événement se lance quand le robot a fini de se déplacer.

Chef Elementaire est assez simple à comprendre par contre pour l'autre on a cet enchaînement d'évènements:

